CloudVault — Step-by-step build guide

FastAPI · PostgreSQL on Supabase · Supabase Storage · React

Updated implementation guide — Supabase Storage, private buckets, custom JWT, Alembic, real integration tests

Decisions fixed in this edition

• Production bucket name: **cloudvault**. Test bucket name: **cloudvault-test**.

• File bytes never use an application-managed local storage directory. FastAPI may temporarily spool multipart uploads through Starlette; this is not durable storage.

• The backend is the only Supabase client. It uses custom JWT authentication, the protected database connection, and the service-role key only on the server.

• Alembic is the only schema migration system. RLS is enabled with no Data API policies in v1.

• Storage integration tests use a dedicated private Supabase test bucket and refuse production settings.

• Upload and delete flows explicitly handle failures across PostgreSQL and Supabase Storage.

1. Architecture and invariants

Supabase provides PostgreSQL and private object storage. FastAPI owns authentication, validation, ownership checks, quota accounting, sharing, and response headers. React calls FastAPI only; it never receives the service-role key and never calls Supabase directly.

Non-negotiable invariants

• Every database file row has exactly one private Storage object, except briefly during a controlled upload/delete failure window.

• The storage key is server-generated: **{numeric_user_id}/{32_hex_characters}.bin**. Original names never become object keys.

• A user cannot access another user's rows or objects. Foreign private resources return 404.

• used_bytes equals SUM(files.size) for each user.

• A view share is allowed only for PDF, PNG, JPEG, or TXT. ZIP is download-only.

• The production bucket is **cloudvault**; the integration bucket is **cloudvault-test**.

2. Decisions before coding

2.1 Environment and bucket setup

Create two private buckets in Supabase Storage. Disable public access and create no anon/authenticated storage policies. The service-role key remains backend-only. Confirm the plan and bucket limit support at least 100,000,000 bytes per object; some plans cap uploads at 50 MB.

```
SUPABASE_URL=https://YOUR_REF.supabase.co
SUPABASE_SERVICE_ROLE_KEY=server-only-secret
SUPABASE_BUCKET=cloudvault
TEST_SUPABASE_BUCKET=cloudvault-test
MAX_UPLOAD_BYTES=100000000
DEFAULT_QUOTA_BYTES=2000000000
```

Use the session pooler connection string with postgresql+psycopg://, URL-encoding special characters in the password. Never commit .env or print settings in logs.

2.2 Transaction strategy

Do not hold a database row lock while uploading a slow request to Storage. Validate and spool the request first, reserve quota in a short transaction, upload to Supabase, then insert metadata in a second transaction. If metadata insertion or commit fails, delete the object and release the quota reservation. A reservation table makes recovery explicit.

```
upload_reservations(id, user_id, storage_key, size, created_at, status)
statuses: reserved | committed | released
```

The reservation row is optional for a minimal implementation, but the cleanup job must understand in-progress objects and must not delete active uploads. A one-hour grace period is required for objects without metadata.

3. Project setup

```
mkdir CloudVault && cd CloudVault
git init
mkdir backend && cd backend
python3 -m venv venv
source venv/bin/activate
pip install fastapi uvicorn sqlalchemy psycopg[binary] alembic python-multipart pyjwt pwdlib[argon2] pydantic-settings em
pip install pytest
mkdir -p app/models app/schemas app/routers app/services app/utils tests
```

Create .gitignore entries for .env, venv, node_modules, __pycache__, and dist. Do not create a storage/ directory. The application must not persist file bytes locally.

Required configuration

```
DATABASE_URL=postgresql+psycopg://...
SUPABASE_URL=https://REF.supabase.co
SUPABASE_SERVICE_ROLE_KEY=...
SUPABASE_BUCKET=cloudvault
TEST_SUPABASE_BUCKET=cloudvault-test
SECRET_KEY=...
ACCESS_TOKEN_MINUTES=60
MAX_UPLOAD_BYTES=100000000
DEFAULT_QUOTA_BYTES=2000000000
FRONTEND_ORIGIN=http://localhost:5173
```

4. Database and migrations

Create users, folders, files, shares, and—preferably—upload_reservations. Use composite ownership foreign keys, partial unique indexes for root/child names, non-negative checks, and ON DELETE CASCADE only for shares. Import every model in app/models/__init__.py.

Run Alembic against Supabase. Review every generated migration manually; autogenerate may miss check constraints, partial index predicates, RLS, and storage-related tables. Enable RLS on users, folders, files, and shares in the migration. Create no anon/authenticated policies in v1 because the backend is the only database client.

```
alembic revision --autogenerate -m "day1 schema"
# review and complete the migration
alembic upgrade head
```

Never use create_all for production. Test migrations separately on a disposable database. Do not run downgrade base against a database containing real data.

5. Day 1 — Authentication

Implement custom JWT authentication only. Do not add Supabase Auth. Normalize email to lowercase, use Argon2id through pwdlib, verify a dummy hash for unknown emails, pin the JWT algorithm, and require exp and sub claims.

Checkpoint

Register, duplicate-register, login, wrong-password, unknown-email, /me, expired-token, and tampered-token tests pass. Wrong-password and unknown-email responses must have the same status and JSON body.

6. Day 2 — Supabase Storage service

Create a SupabaseStorage interface with upload, download, delete, exists, list, and create_signed_url operations. Implement it using httpx against the Supabase Storage REST API. Validate the bucket name and object key before every call. Use bounded timeouts and map upstream failures to STORAGE_UNAVAILABLE (502); do not expose upstream responses or service credentials.

6.1 Upload algorithm

```
1. Authenticate and resolve the destination folder by ownership.
2. Sanitize the display filename and validate its extension.
3. Read the framework-spooled upload in 1 MiB chunks; count, hash, and retain the first 8 KiB.
4. Reject over 100,000,000 bytes and content mismatches.
5. Reserve quota atomically in PostgreSQL.
6. Generate {user_id}/{uuid4().hex}.bin.
7. Upload the file object to private bucket cloudvault.
8. Insert the metadata row and commit.
9. On any failure after upload, delete the object and release the reservation.
10. Return metadata without storage_key.
```

The client Content-Type is ignored. Send the server-assigned MIME type and use controlled overwrite behavior. Never retry a non-idempotent upload blindly; retries must reuse a known key and verify the resulting object.

6.2 Download and preview

Perform the ownership or share check first. Then stream through FastAPI or generate a short-lived signed URL. The bucket is never exposed to the browser. Set Content-Disposition using a sanitized filename, media type from validated metadata, Content-Length when known, X-Content-Type-Options: nosniff, and no-store for public shares.

Checkpoint

Upload a valid PDF to cloudvault, verify the object and metadata row, download it through FastAPI, verify checksum, test a private direct bucket URL is denied, and test a short-lived signed URL before and after expiry.

7. Day 2 — File operations and quota

Implement list, details, rename, delete, search, type filtering, sorting, pagination, and usage. Whitelist sort fields and escape ILIKE wildcards. Never return storage_key.

7.1 Delete algorithm

Delete metadata and decrement usage in a database transaction first. After commit, delete the corresponding object. If deletion fails, log an internal cleanup job and let orphan cleanup retry it. A missing object is treated as already deleted. Recursive folder deletion must collect keys, delete rows, update usage once, commit, and then delete objects.

7.2 Quota concurrency

Use an atomic conditional UPDATE or reservation row. Test two parallel uploads whose combined size exceeds the quota; exactly one must succeed. Do not compute quota from a bucket scan.

Checkpoint

Run tests for duplicate names, cross-user access, path-like filenames, invalid content, quota overflow, concurrent uploads, failed metadata commit, failed object delete, checksum, usage invariant, and 95,000,000/101,000,000-byte files.

8. Day 3 — Folders and dashboard

Implement nested folders, breadcrumbs, moving files, rename, non-empty deletion protection, recursive deletion, all-file search, filters, sorting, pagination, and usage endpoint. Add an all=true option for dashboard recent files so files inside folders are included.

Checkpoint

Test composite foreign keys, cross-user folder moves, root/child duplicate names, recursive deletion and usage correctness.

9. Day 4 — Sharing

Generate secrets.token_urlsafe(32), store only its SHA-256 hash, and return the raw token once. Require expiry, cap it at 30 days, support view/download permissions, optional Argon2 password protection, and revocation.

Public access sequence

```
1. Hash token and find the share.
2. Return the same 404 for missing, expired, revoked, deleted, or missing-object shares.
3. Verify X-Share-Password when configured.
4. Re-check permission and preview type.
5. Stream through FastAPI or create a short-lived URL.
6. Set nosniff, no-referrer, and no-store.
```

A ZIP view share must return 422 at creation. A ZIP download share may return 201. A signed URL must never outlive the effective share policy; check the share immediately before generating it and use a short lifetime.

Checkpoint

Test valid, expired, revoked, deleted-file, missing-object, password, wrong-password, view/download disposition, ZIP rejection, rate limiting, and cross-user share access.

10. Day 5 — Cleanup and hardening

Implement hourly cleanup. List the private cloudvault bucket with pagination, query known storage_key values in batches, and delete only objects older than the one-hour grace period that have no matching file row or active reservation. Log failures internally and retry them. Never log tokens, passwords, signed URLs, service keys, or raw object keys in user-facing messages.

Security headers and deployment

Add CORS restricted to FRONTEND_ORIGIN, nosniff, Referrer-Policy, Cache-Control for public content, strict input validation, rate limiting on login/register/public shares, HTTPS in production, and a Content-Security-Policy where the frontend permits. localStorage JWTs remain a version-one tradeoff; use secure HttpOnly cookies for production.

11. Testing strategy

Use a dedicated local PostgreSQL database for database tests and the dedicated private Supabase bucket cloudvault-test for Storage integration tests. The suite must refuse to start if the database URL points at Supabase or the bucket is not exactly cloudvault-test. Never use the production bucket.

Required test layers

• Unit tests for validation, key generation, MIME rules, error mapping, and filename header sanitization.

• Database tests against local PostgreSQL for constraints, ownership, quota, transactions, and migrations.

• Supabase integration tests against cloudvault-test for upload, object existence, private access, signed URLs, deletion, pagination, and cleanup.

• API tests for all status codes, response shapes, headers, cross-user access, and share behavior.

• A cleanup fixture deletes every test object by the test-user prefix after each test, even when assertions fail.

Required safety checks

```
assert settings.SUPABASE_BUCKET == "cloudvault"
assert settings.TEST_SUPABASE_BUCKET == "cloudvault-test"
assert "supabase" not in TEST_DATABASE_URL.lower()
assert not settings.SUPABASE_SERVICE_ROLE_KEY.startswith("sb-") or True  # never print or commit it
```

Use an isolated test user prefix and deterministic cleanup. Do not use Base.metadata.drop_all against any Supabase database. Apply migrations only to disposable test databases.

12. Frontend

React uses one Axios instance pointing to FastAPI. It never imports Supabase client libraries. Uploads use FormData without manually setting Content-Type, show progress and a Processing state, and refresh file lists and usage after completion. Preview/download responses are backend blobs; revoke object URLs. Disable View sharing for non-previewable files including ZIP.

13. Completion checklist

Foundation: environment settings, Alembic schema, RLS enablement, custom JWT, protected routes. Storage: private cloudvault bucket, SupabaseStorage service, validation, checksums, atomic quota, upload rollback, streaming download, signed URL policy, cleanup. Files/folders: CRUD, ownership, search, filters, moves, recursive deletion. Sharing: hashed tokens, expiry, password, revocation, ZIP download-only. Quality: real cloudvault-test integration suite, migration tests, concurrency tests, cleanup tests, rate limiting, security headers, frontend loading/error states, and README.

14. Troubleshooting and operational rules

Storage unavailable

Return 502 STORAGE_UNAVAILABLE, keep the metadata and quota consistent, and do not return a success-shaped response. Retry only safe/idempotent operations with bounded backoff.

Object exists but metadata is missing

Leave it until the grace period expires, then cleanup removes it unless an active reservation exists. Investigate repeated occurrences through internal logs/metrics.

Metadata exists but object is missing

Return 404 for download, log the inconsistency, and expose an admin repair/reconciliation path rather than silently recreating or changing metadata.

Final implementation rule

The revised blueprint and this guide must use the same bucket names, storage adapter contract, transaction behavior, environment variable names, and test expectations. Any deviation should be written down before coding so deployment and tests do not drift.